

On the Distribution of DTN Reachability Information

A quantitative **push-vs-pull** analysis. A bundle carries the name `ipn:13.1` and nothing else. Somebody has to know where that is. Should that knowledge be **flooded to everyone in advance**, or **asked for when needed**? The answer turns out to be set by a single variable.

Juan A. Fraire — Inria, INSA Lyon, CITI, UR3720, Villeurbanne, France · D3TN GmbH, Dresden, Germany
Funded by the DARKSOL project · models and configurations are open

The question hiding under every bundle

A bundle carries a name. BPv7 addresses an *endpoint identifier* — `ipn:13.1` — and nothing more.

A radio needs an address. Before that bundle moves one hop, the node must turn the name into a convergence-layer endpoint: which CLA, which address, which port.

The standards left the arrow blank. RFC 4838 deliberately does not specify how the EID-to-CLA binding is distributed. BPv7 and the CLAs assume it is already there.

Every other layer of DTN has a protocol. The binding between a name and a way to reach it has an operator.

This is the exact analogue of what IP routing and DNS do for the Internet — and in DTN it is currently done by hand.

Today, it is a file somebody maintains

```
# static EID → CLA table, hand-written, per node
ipn:13.1   → tcpclv4://10.0.4.7:4556
ipn:14.1   → ltp://mars-relay.example:1113
ipn:15.*   → udpcl://192.168.9.3:4556
```

- Fine for five nodes and one operator: you write the file once and it is correct.
- **Every new asset** is an edit on every node that might ever talk to it.
- **Every re-binding** — a rover swaps radios, a relay is retasked, a gateway moves — is that edit again.
- When it is wrong, nothing announces it. Bundles are simply never delivered.

Static provisioning does not fail loudly. It fails silently, and it fails more often as the network grows.

Why this stops working now

Cislunar buildout

LunaNet and Artemis: dozens of assets — orbiters, surface bases, rovers, EVA — arriving and retiring across a decade, operated by several agencies at once.

Multi-operator by construction

No single authority owns the table any more. The node that needs the binding and the node that owns it belong to different organizations.

Mobility and churn

Assets are re-tasked, radios are swapped, relays change roles. The binding is not a constant; it is a value with a lifetime.

The scale at which hand-provisioning breaks is arriving. The mechanism that replaces it will be chosen once, and then frozen into deployments for a very long time.

This is a question worth answering before it is answered by default.

Two proposals landed in 2025 — from opposite directions

Push it: BGP-like

Feldmann et al., WiSEE 2025. Extend BGP UPDATE messages to carry DTN EID reachability, reusing path-vector machinery that already exists and is operationally understood.

Pull it: DNS-like

Kline, draft-ek-dtn-ipn-arpa-00, Nov 2025. An `ipn.arpa` zone that resolves `ipn`-scheme EIDs through the DNS hierarchy the Internet already runs.

- Both are technically sound. Both answer **the same question**. Their cost structures are opposites.
- Push pays $O(N)$ messages per change to make every lookup free. Pull pays $O(1)$ per registration and $2 \cdot D \cdot d$ per lookup, forever.
- Nobody had compared them across the delay range DTN actually spans.

That comparison is this paper.

The Internet already answered this — twice

BGP: push everything now

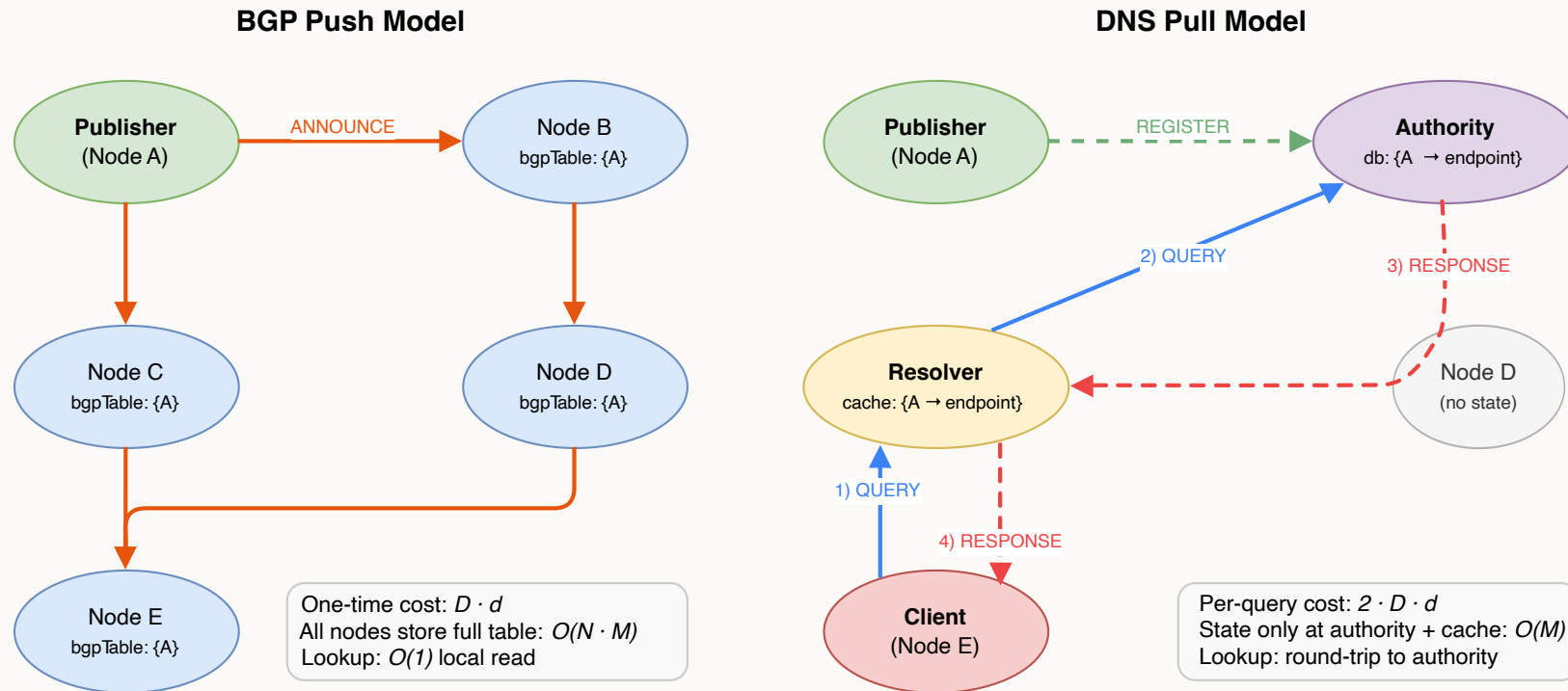
Announce every prefix to every peer, flood until all speakers agree. Lookup afterwards is a local table read. Strong consistency, and a table that lives everywhere.

DNS: pull only when needed

Keep the record at an authority, walk the hierarchy on demand, cache for a TTL. Almost no background traffic, and a round trip on every miss.

- On the Internet these are **complementary**: routes are pushed, names are pulled, and they run side by side solving different problems.
- In DTN, one question — *where is `ipn:13.1`?* — can be answered by either paradigm.
- So what we compare is not two protocols competing for a job. It is **two ways of moving the same fact**, applied to one job.

Push and pull, side by side



Left: flooding replicates every binding to every node — one-time cost $D \cdot d$, network state $O(N \cdot M)$, lookup free. **Right:** resolution keeps bindings at an authority (grey nodes hold no state) and fetches on demand — $2 \cdot D \cdot d$ per query, state $O(M)$.

The two cost structures are mirror images

	Push (BGP-like)	Pull (DNS-like)
Publish one binding	$O(N)$ messages, network-wide flood	$O(1)$ — tell the authority
Look one up	local read, ≈ 0	$2 \cdot D \cdot d$, every time
State in the network	$O(N \cdot M)$ — full replication	$O(M)$ — one copy plus caches
A binding changes	flood again	update the authority; caches go stale
Is the answer current?	yes, after convergence	as fresh as the TTL allows

Push pays once, everywhere. Pull pays every time, only where it is needed. The exchange rate between those two costs is the link delay.

Delay is what decides the answer

Terrestrial · $d \approx 10$ ms

A pull query costs about **0.25 s** — nobody notices. The bandwidth saved by not flooding is pure profit.

Earth–Moon · $d = 1.3$ s

A pull query across the trunk costs about **2.8 s**. Push converges once in ~ 1.6 s. Now it depends on how often you ask.

Earth–Mars · $d = 720$ s

A pull query costs up to **25 minutes**. Push converges once and every later lookup is free.

- The one-way delay in a solar-system network spans **five orders of magnitude**, from milliseconds to twelve minutes.
- Same two protocols, same topology, same workload — and somewhere in that range the right answer inverts.

The question is not “which is better”. It is “where does it flip, and by how much”.

What we built

Two OMNeT++ models, one topology

Push: path-vector flooding — path vectors, loop detection, per-EID versions, withdrawals. **Pull:** authorities, caching resolvers, TTL expiry, registration and de-registration.

A fair comparison, deliberately

Pull runs over a *pre-routed underlay* ($RTT = 2 \cdot D \cdot d$) — real DNS runs over a routed IP network, and we refuse to charge it for a routing problem that is not the question.

- A central **GroundTruth** module knows the true binding at every instant, so correctness is measured, not inferred.
- **13 experiments:** grids from 25 to 400 nodes, 10 to 500 EIDs, links from 10 ms to 20 s, plus three realistic deployments.
- Every configuration repeated **3–10 times** with independent seeds.
- Models, `.ini` files and analysis scripts are public: github.com/darksol-cloud/bgp-dns-omnetpp

Four hypotheses, written down before the sweeps

H1 — Pull wins at low delay. When $2 \cdot D \cdot d < 1$ s, its $O(1)$ publication and churn cost give strictly lower overhead.

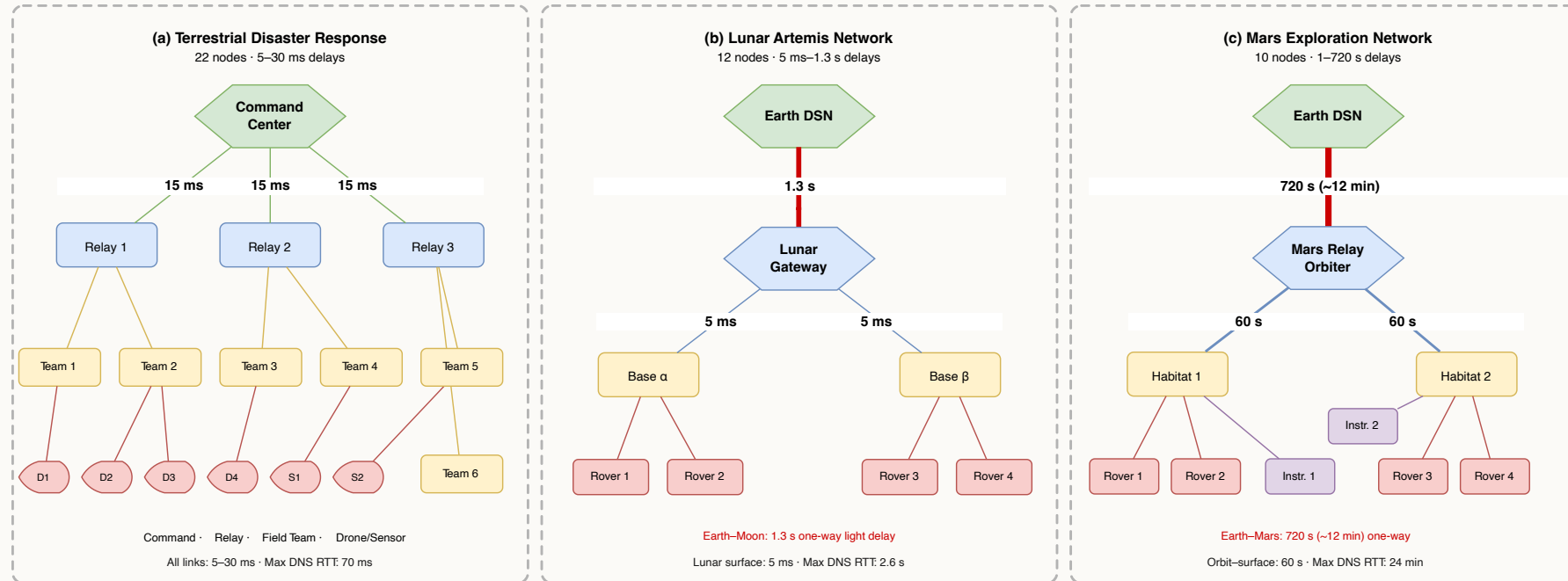
H2 — Push wins at high delay. Once $2 \cdot D \cdot d$ reaches tens of seconds, cumulative pull cost passes one-time push convergence after very few queries.

H3 — There is a crossover near cislunar delay (~1–2 s one-way), where pull still saves bandwidth while push already gives lower amortized latency.

H4 — Pull accuracy degrades with $R = T_{\text{conv}} / \Delta_{\text{churn}}$. When the network cannot converge between changes, caches serve stale answers. Push does not care.

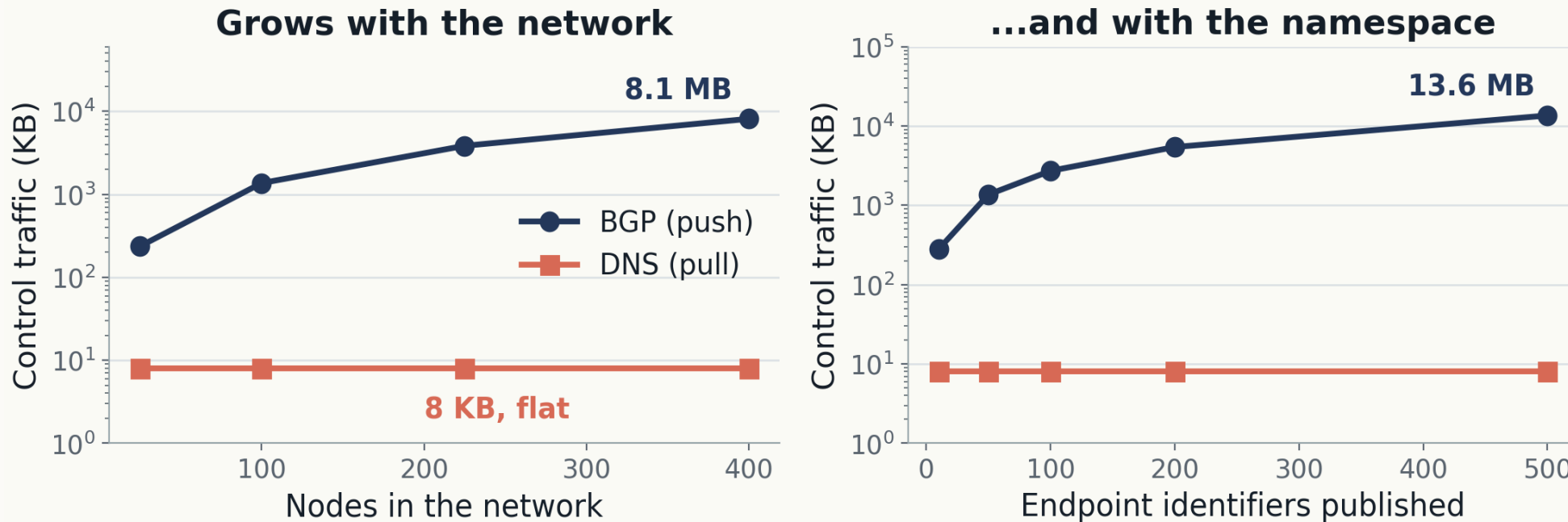
All four were confirmed. The interesting part is *where* the boundaries fall and how sharp they are.

Where we ran them



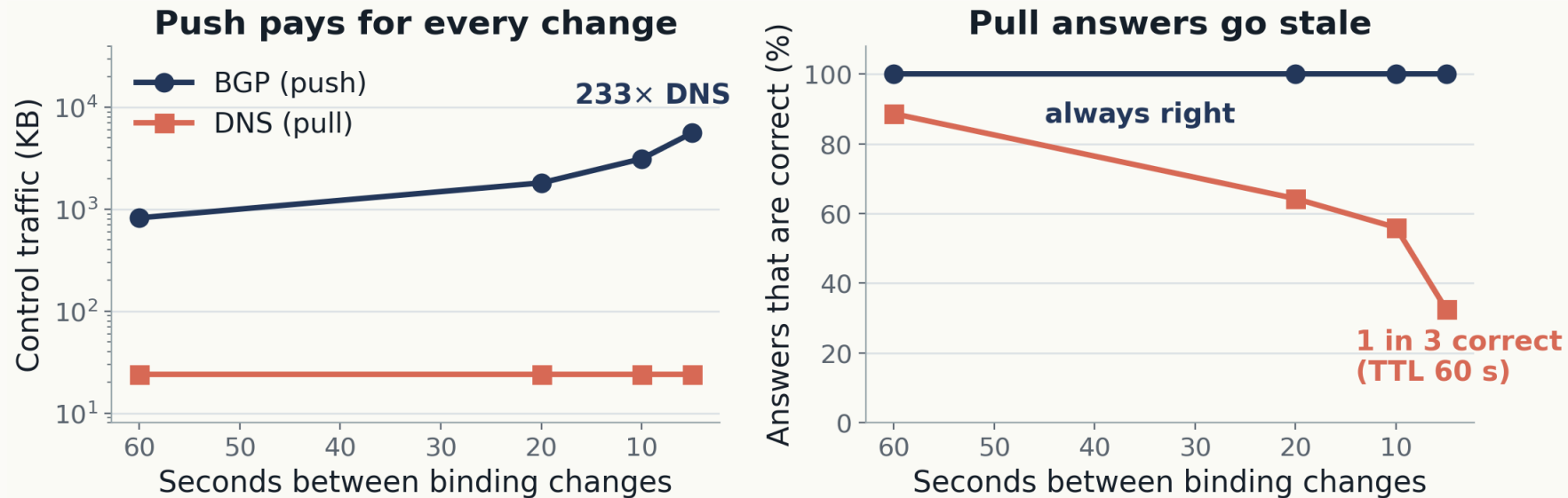
Three deployments spanning the whole spectrum: **terrestrial disaster response** (22 nodes, 5–30 ms), **lunar Artemis** (12 nodes, the 1.3 s Earth–Moon trunk dominates), **Mars exploration** (10 nodes, a 720 s interplanetary link over 60 s orbit-to-surface hops). Plus synthetic grids for controlled sweeps.

Result 1 — push does not scale quietly



- 25 → 400 nodes: **236 KB → 8.1 MB**, a 34× rise for 16× the nodes. Flooding in a grid is super-linear.
- 10 → 500 EIDs on a fixed grid: **280 KB → 13.6 MB**, about 27 KB of control traffic per name published.
- Pull stays at **8 KB**, because its cost depends on how often you ask, not on how large the network is.

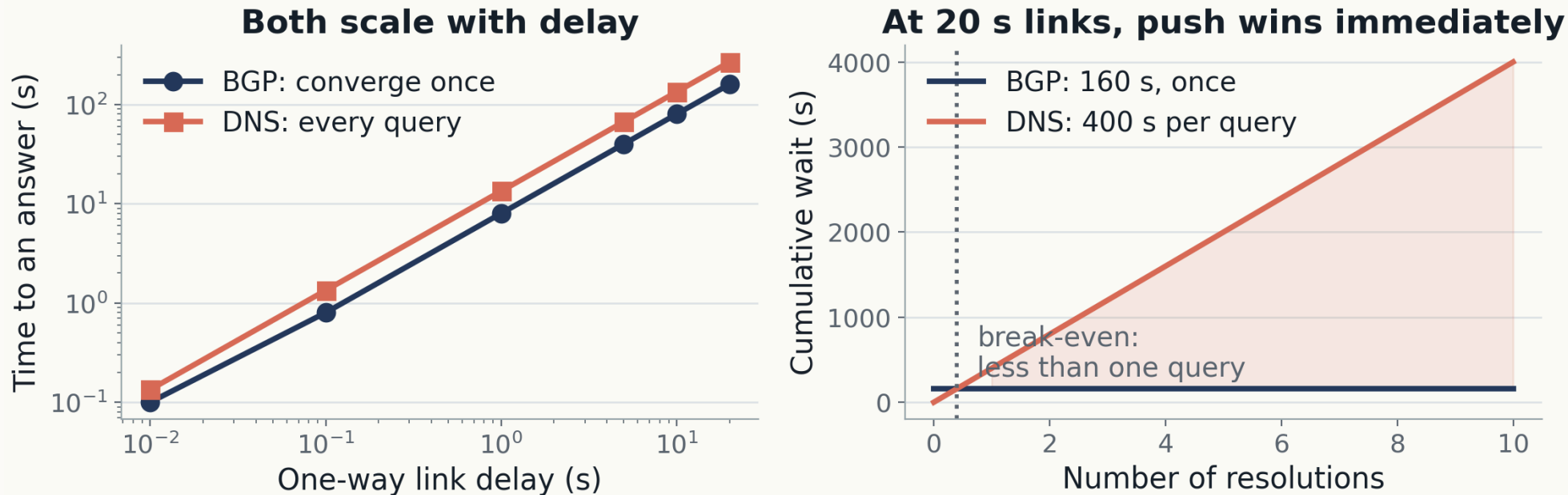
Result 2 — pull is cheap because it may be wrong



- Every binding change re-floods: at a 5 s churn interval, **5.6 MB against 24 KB** — a factor of 233.
- Push answers stay current. Pull answers are only as fresh as the TTL: 60 s TTL against 5 s churn leaves **one answer in three correct**.
- So the TTL has to track the churn interval: 15 s TTL gives 83%, 120 s gives 35%, under identical churn.

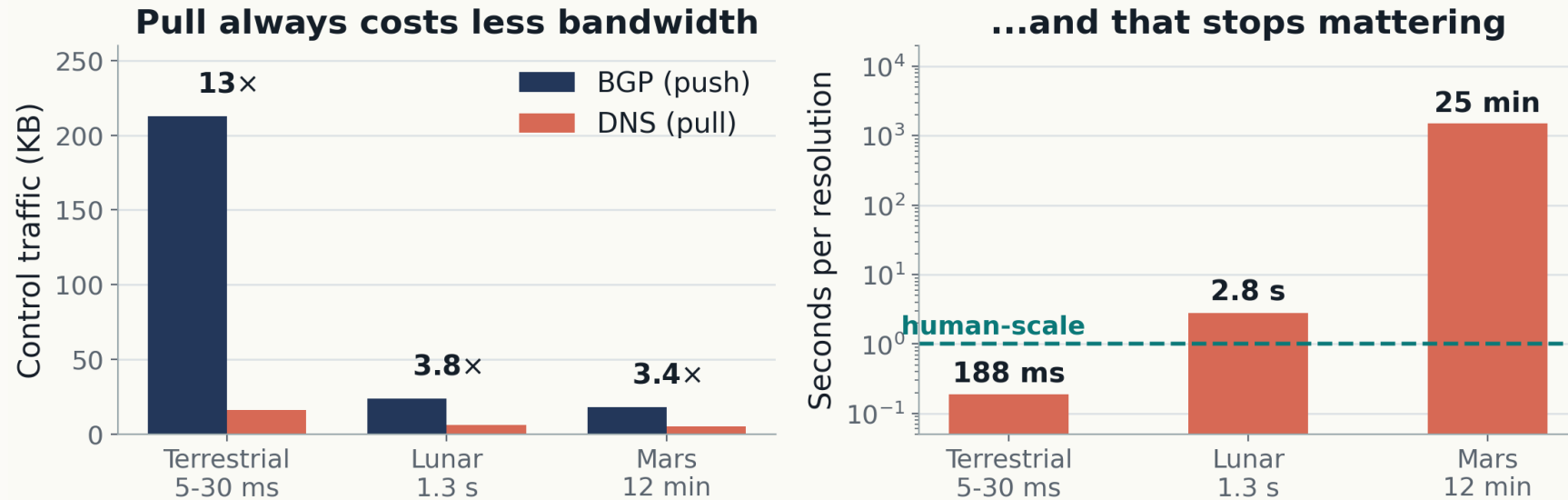
The bandwidth pull saves is paid for, in part, with staleness. Choosing pull means choosing an accuracy budget.

Result 3 — past a few seconds, delay flips the answer



- Both costs grow linearly with the link delay. The difference is **how often you pay**.
- At 20 s links: push converges once in 160 s and every lookup afterwards is free; pull is 400 s, per query, forever.
- Break-even at that distance is **less than a single query** — the first resolution already justifies having flooded.
- Caching helps and does not rescue it: 45% hit rate, 41% lower mean latency, still 235 s per resolution.

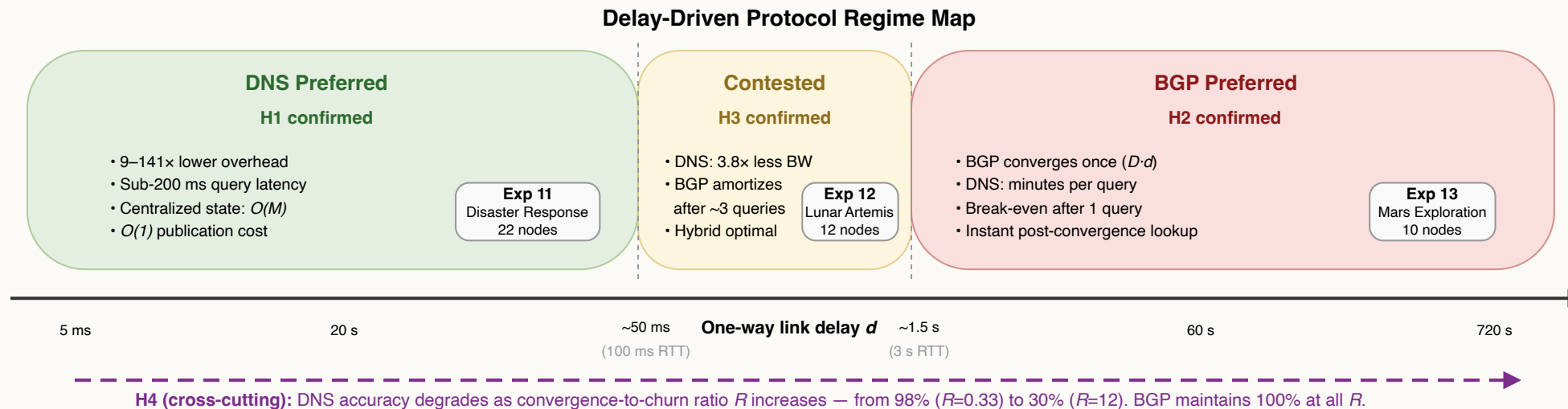
Three deployments, two dimensions



- **Terrestrial:** pull uses 9–13x less traffic and answers in 188 ms. Pull is the right choice.
- **Lunar:** pull still saves 3.8x, but push amortizes after about **3 queries**. Genuinely contested.
- **Mars:** pull saves 3.4x of a tiny number, and costs 25 minutes per Earth-involved answer.

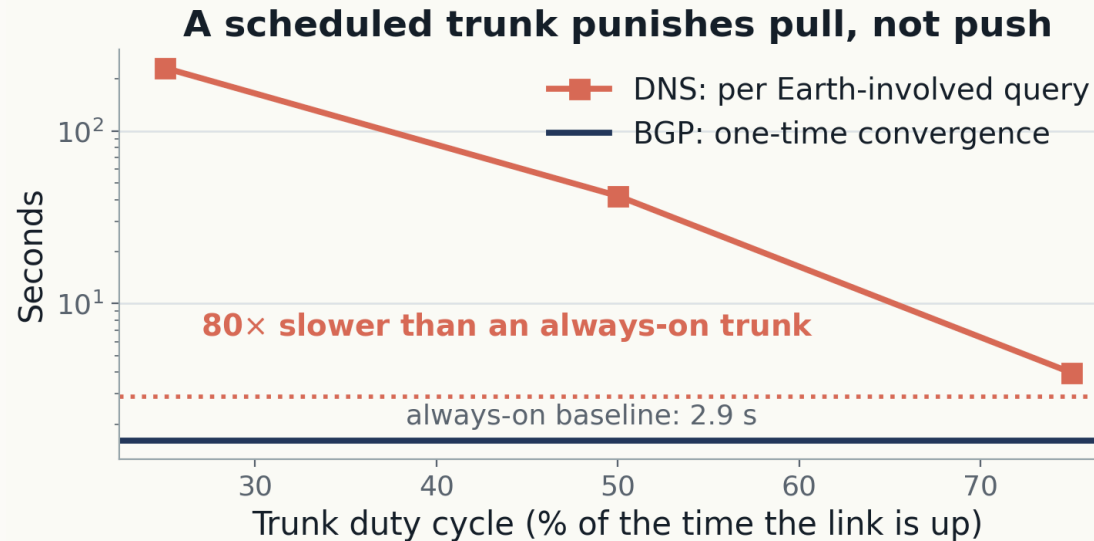
Pull's bandwidth advantage stays roughly constant. Its latency cost moves five orders of magnitude. That asymmetry is the result.

The deliverable: a delay-driven regime map



Below ~100 ms RTT pull wins on overhead (9–141×) at latency nobody notices. **Above ~3 s** push is the only viable option. **In between** the band is contested and a hybrid is indicated: push for anything queried across the trunk, pull for what stays local. Cross-cutting, pull accuracy falls as R grows; push is unaffected.

The caveat we can quantify: links are scheduled



- Everything so far assumes links that are **up**. Real space links are duty-cycled and contact-scheduled.
- A flooded update **waits once** at a closed trunk, then propagates. A pull query **waits again, every time**.
- Lunar trunk at 25% duty: Earth-involved queries go 2.9 s → 230 s, while push overhead is byte-identical and accuracy stays at 100%.

Intermittency moves the contested band toward push. The always-on results here are the best case for pull, not the average one.

What to take away

1. **The binding layer needs a mechanism, not an operator.** It is the last hand-maintained table in an otherwise automated stack.
2. **One-way delay is the single most predictive variable** for choosing that mechanism — more than node count, more than churn rate.
3. **The rule of thumb:** pull below ~100 ms RTT · push beyond ~3 s · hybrid in between · and if you pull, make the TTL track the churn interval.
4. **Scope, stated honestly.** Our push results characterize *generalized multi-hop flooding*, not the edge gateway-to-BPA configuration that the BGP extension originally targeted.
5. **Next:** a concrete hybrid for the cislunar band, hierarchical authorities, contact-plan-aware resolution, and the cost of securing it with BPSec.

Paper, models, all 13 experiment configurations and the analysis scripts: github.com/darksol-cloud/bgp-dns-omnetpp